

# When Should Global Guidance Be Refreshed? A Paired Pareto Study in Lifelong Multi-Agent Path Finding

Anonymous Author(s)

## ABSTRACT

Lifelong multi-agent path finding (LMAPF) requires robot fleets to serve an ongoing stream of goals while avoiding congestion. Learned global guidance can improve traffic flow, but existing online methods commonly refresh guidance at fixed intervals. Frequent refresh consumes generator calls even when demand is stable, whereas infrequent refresh may leave guidance stale after workload shifts. We formulate refresh timing as a budgeted control problem and introduce Context-Aware Refresh and Recall (CARR), which causally chooses to retain active guidance, generate a new graph, or reactivate a previously generated one using demand change, context similarity, guidance age, and route-adoption signals. CARR wraps a frozen generator and planner without retraining either. We evaluate eight refresh policies on stationary and non-stationary warehouse workloads using paired, policy-independent task streams and an audit-complete protocol. The pre-specified primary objective is not met: dense refresh retains a small but measurable mean-throughput advantage (CARR effect  $-0.80\%$ ; 95% CI  $[-1.19\%, -0.40\%]$ ), so 1% non-inferiority is not established. In pre-specified secondary analyses, CARR uses about 79% fewer generator calls, forms a non-dominated sample-mean throughput-call operating point, and significantly outperforms all five evaluated low-call comparators. Our results identify guidance timing as an independent systems-design dimension and provide a reproducible basis for choosing refresh policies according to resource constraints.

## CCS CONCEPTS

• **Computing methodologies** → **Multi-agent systems**; *Motion path planning*.

## KEYWORDS

lifelong multi-agent path finding, multi-robot coordination, adaptive global guidance, resource-aware planning, event-triggered refresh, non-stationary workloads, Pareto analysis

## ACM Reference Format:

Anonymous Author(s). 2026. When Should Global Guidance Be Refreshed? A Paired Pareto Study in Lifelong Multi-Agent Path Finding. In *Proceedings of The 8th International Conference on Distributed Artificial Intelligence (DAI 2026)*. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/nnnnnnn>.

## 1 INTRODUCTION

Lifelong multi-agent path finding (LMAPF) asks a fleet of agents to serve an ongoing stream of goals without collisions while maintaining throughput [7]. Scalable solvers often separate a shared,

slowly varying coordination signal from fast local motion decisions. Direction maps, highways, guide paths, and learned guidance graphs all bias agents away from congested interactions [1, 2, 5, 12]. OnlineGGO advances this idea by conditioning guidance on recent traffic and task distributions, but its update interval still controls a stated trade-off between adaptability and recomputation [10].

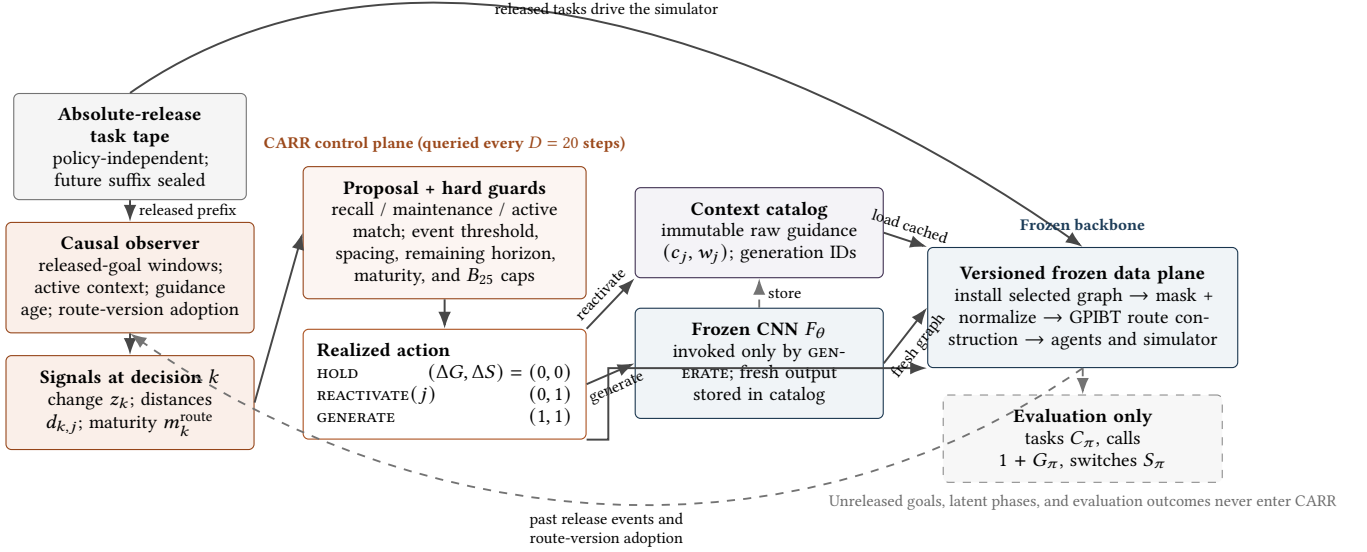
This leaves an orthogonal control-plane question: *given a fixed guidance generator and planner, when is another generator invocation warranted?* Periodic refresh ignores whether demand has changed. Refreshing at every opportunity consumes calls even when the active artifact remains appropriate; refreshing rarely can retain stale traffic preferences after a workload shift. A previous graph may also become relevant again when demand recurs. The resulting decision is not simply whether guidance is useful, but how much throughput each additional invocation buys.

We study this question in a frozen OnlineGGO-style CNN+GPIBT system. Our controller, Context-Aware Refresh and Recall (CARR), chooses among holding the active graph, generating a new graph, and reinstalling a historical graph. It observes only causally available endpoint histograms, event history, guidance age, and route-adoption maturity. The generator, downstream planner, simulator, and evaluation task tapes are identical for every policy. We ask:

- **RQ1:** Does context-aware refresh improve throughput over no online refresh and weak sparse schedules?
- **RQ2:** What throughput-call trade-off does it achieve relative to dense periodic refresh?
- **RQ3:** Does historical reactivation add throughput or primarily substitute for fresh generations?

Our contributions are threefold. First, we formulate guidance publication as a budgeted, causal control problem and instantiate CARR with explicit generation, installation, and delayed-adoption semantics. Second, we provide a fresh-root, paired evaluation of eight publication policies: 3,840 isolated runs over forty independent root clusters, externally fixed task tapes, root-cluster inference, multiplicity control, and an auditable no-deletion ledger. Third, we report the failed primary non-inferiority result together with its pre-specified secondary trade-off analyses. CARR uses 5.46 calls on average, lies on the empirical sample-mean Pareto frontier, and significantly outperforms all five frozen low-call comparators after correction; historical recall shows no detected throughput benefit. We therefore make a bounded resource-throughput claim within one frozen backbone, not a global LMAPF state-of-the-art claim.

Figure 1 separates the frozen LMAPF data plane from CARR's sole intervention: a causal, budgeted decision over guidance publication. This separation is central to our attribution claim. Any change in a paired rollout must arise from which already-defined guidance operation is executed, rather than from retraining the generator, modifying GPIBT, or changing the exogenous task stream.



**Figure 1: CARR’s causal control plane around a frozen guidance generator, GPIBT planner, and simulator. Past release and route-adoption events produce the change, context, age, and maturity signals used to hold, reactivate, or generate guidance under hard guards. Recall loads an immutable catalog entry without invoking the CNN; generation calls the CNN and stores a new entry. Installed versions reach agents only at route construction, and evaluation outcomes do not feed back to the controller.**

## 2 RELATED WORK

### 2.1 Lifelong multi-agent planning

Early formulations of lifelong or online multi-agent routing include pickup-and-delivery task streams and token-passing coordination [7]. PIBT instead constructs agents’ next actions using priority inheritance and backtracking, with finite-time guarantees under specific graph conditions [8]. RHCR repeatedly solves bounded-horizon MAPF instances and represents a complementary replanning-based family [6]. We do not empirically compare these algorithm families. Our study fixes a GPIBT-style downstream planner so that differences can be attributed to guidance publication policy.

### 2.2 Global traffic guidance

Shared directional preferences have progressed from learned direction maps and user-specified highways [2, 5] to congestion-avoiding guide paths [1]. Guidance Graph Optimization (GGO) directly optimizes edge weights or a model that produces them [12]; later work extends the representation to mixed edge directions and weights [11]. OnlineGGO dynamically generates guidance from recent traffic and current task distributions and integrates it with both direct and guide-path planning [10]. These works primarily ask what guidance should contain. We keep the learned generator fixed and ask when its output should be retained, regenerated, or recalled.

### 2.3 Adaptive update timing

Update timing is an established control variable outside LMAPF. Event-triggered control replaces strictly periodic execution with state-dependent events [9]; navigation work has studied learned replanning timing and path-regeneration decisions in dynamic environments [3, 4]. We borrow the resource-aware motivation, not their stability or navigation guarantees. Our setting differs in that a shared guidance artifact influences many agents, adoption is delayed until route construction, and evidence is evaluated as a paired throughput-call frontier.

## 3 PROBLEM SETTING AND SYSTEM SEMANTICS

### 3.1 Lifelong routing with fixed guidance generation

Let  $G = (V, E)$  be a directed grid graph and let  $n$  agents repeatedly receive sequential goal-reaching tasks. At each simulator step an agent waits or moves to an adjacent vertex; vertex conflicts and opposing edge swaps are forbidden. The planner consults a shared directed-edge guidance tensor while constructing routes.

A frozen convolutional generator  $F_\theta$  maps the causal traffic observation at decision  $k$  to a four-channel raw edge-weight tensor,

$$w_k = F_\theta(o_{\leq t_k}) \in \mathbb{R}^{4 \times H_m \times W_m}. \quad (1)$$

The environment masks invalid edges and normalizes  $w_k$  before the GPIBT backend consumes it. We vary only the policy that decides whether to hold, replace, or recall guidance. The checkpoint, planner, simulator, maps, and task tapes remain fixed.

Each episode contains a 200-step warm-up and a scored horizon of 2,000 steps. Guidance decisions occur every  $D = 20$  steps, producing  $N = 100$  scored windows. All policies make one mandatory bootstrap call. Of the 99 post-bootstrap decisions, the focal ceiling is

$$B_{25} = \lceil 0.25(99) \rceil = 25. \quad (2)$$

For policy  $\pi$ , let  $C_\pi$  be scored task completions,  $G_\pi$  fresh post-bootstrap generator calls, and  $S_\pi$  effective post-bootstrap guidance installations. We study the two-dimensional operating point ( $\mathbb{E}[1 + G_\pi], \mathbb{E}[C_\pi]$ ) and report installations separately. Generator calls are a pre-specified logical resource unit; they are not a proxy for a measured wall-clock, energy, or monetary reduction.

### 3.2 Publication-control objective

At decision  $k$ , a publication controller has a causal history  $\mathcal{H}_k$  containing only observations exposed by time  $t_k$ , an active generation, and (for memory policies) a catalog of earlier generations. It maps this state to one of three operations. Define the per-decision resource indicators

$$g_k = \mathbb{I}[a_k = \text{GENERATE}], \quad s_k = \mathbb{I}[a_k \neq \text{HOLD}]. \quad (3)$$

The two counts are deliberately distinct. Generating installs a new artifact and therefore increments both; reactivation installs an old artifact and increments only  $s_k$ ; holding increments neither. Excluding the mandatory bootstrap, the focal policy obeys

$$\sum_{k=1}^{N-1} g_k \leq B_{25}, \quad \sum_{k=1}^{N-1} s_k \leq B_{25}. \quad (4)$$

We do not collapse throughput and calls into a hand-selected scalar reward. Such a scalar would make the conclusion depend on an unmeasured conversion between one generator invocation and one completed task. Instead, the primary analysis separately tests throughput preservation against the dense reference, and the secondary analysis exposes the empirical operating points in call-throughput space. Policy  $A$  point-dominates policy  $B$  only when its sample-mean call count is no larger and its sample-mean completion count is no smaller, with at least one strict inequality. This descriptive frontier is kept separate from inferential pairwise claims.

### 3.3 Policy-independent task tapes

Completion-driven task generation can confound a throughput comparison: a faster policy advances its task random-number stream sooner and may receive a different future workload. We instead materialize complete absolute-release task tapes before timestep zero. For agent  $i$ ,

$$\mathcal{T}_i = ((g_{i,0}, r_{i,0}), \dots, (g_{i,L_i-1}, r_{i,L_i-1})), \quad (5)$$

where  $g_{i,j}$  is a goal and  $r_{i,j}$  its absolute release time. Starts, goals, releases, and identifiers are identical across paired policies and are verified by independent fingerprints. We evaluate stationary,

abrupt, and recurrent endpoint distributions; recurrent demand follows an  $A \rightarrow B \rightarrow A \rightarrow B$  sequence. Phase metadata and unreleased tape suffixes are never controller inputs.

The tape generator staggers each agent’s releases over a common 110-step interval, rather than releasing the whole fleet in a synchronized burst. A stationary tape uses one endpoint-centered distribution throughout. Dynamic tapes change the distribution at scored timesteps 500, 1,000, and 1,500. For the abrupt workload, four distinct endpoint centers are selected deterministically from the root, with adjacent distributions required to have Jensen–Shannon separation of at least 0.30. For the recurrent workload, two such centers alternate as  $A \rightarrow B \rightarrow A \rightarrow B$ . Goals are then sampled once with root-keyed per-agent randomness and sealed into the tape. Consequently, policies may complete different prefixes by the horizon, but they cannot receive different goals merely because one policy finishes earlier. A non-scoring guard suffix ensures that no arm exhausts its finite tape during the measured horizon.

This construction also makes the information boundary testable. At decision  $k$ , the controller sees trace events already released by  $t_k$  and the currently active goals. It does not see the registered workload label, the locations of future phase changes, the selected phase centers, or any unreleased goal. Metamorphic preflight checks altered future suffixes while holding the observable prefix fixed and required the earlier controller trace to remain unchanged.

### 3.4 Generation, installation, and delayed adoption

A *generation ID* names one immutable output of  $F_\theta$ . An *installation version* increments whenever a fresh or historical graph becomes active. Reactivating a graph changes its installation version but not its generation ID. Holding still advances the simulator using a defensive copy of the cached raw tensor.

An installation affects an agent only when the planner constructs a route. Agents already following routes may therefore retain older versions. For active version  $v_k$ , we measure route-adoption maturity as

$$m_k^{\text{route}} = \frac{\#\{\text{active agents whose latest route uses } v_k\}}{\#\{\text{active agents}\}}. \quad (6)$$

This distinction prevents the controller from treating publication as an instantaneous fleet-wide state change.

## 4 CONTEXT-AWARE REFRESH AND RECALL

The paper name CARR corresponds exactly to the evaluated implementation `context_memory_B25`; no post-evaluation behavior change is hidden by the renaming. Its action set is

$$a_k \in \{\text{HOLD}, \text{REACTIVATE}(j), \text{GENERATE}\}. \quad (7)$$

### 4.1 Decision procedure

The controller separates an *operation proposal* from permission to execute it. Let  $R_k$  indicate that an inactive catalog entry is an admissible recall,  $M_k$  indicate scheduled maintenance, and  $A_k$  indicate that the active graph already matches the current context. The

ordered proposal is

$$\tilde{a}_k = \begin{cases} \text{REACTIVATE}(j^*), & R_k, \\ \text{GENERATE}, & \neg R_k \wedge M_k, \\ \text{HOLD}, & \neg R_k \wedge \neg M_k \wedge A_k, \\ \text{GENERATE}, & \text{otherwise.} \end{cases} \quad (8)$$

Thus a matching historical context is considered before maintenance, while a matching active context suppresses an unnecessary generation. Importantly, Eq. (8) does not itself spend a token. If the causal trigger or an execution guard rejects the proposed switch, the realized action is HOLD.

Let  $q_{.75}(z_{<k})$  be the empirical 75th percentile of previously observed change scores. After four history observations, an event is ready when

$$E_k = \mathbb{I}[z_k \geq 0.10] \mathbb{I}[z_k > q_{.75}(z_{<k})]. \quad (9)$$

Maintenance is ready when guidance age is at least 25 windows and  $z_k \leq 0.20$ . A non-hold proposal executes only when either  $E_k$  or  $M_k$  is true and all hard guards pass: at least half of the active fleet has adopted the current installation, six windows have elapsed since the previous switch, at least three effect windows remain, and neither the generation nor switch cap would be exceeded. Formally, with  $\Gamma_k$  denoting their conjunction,

$$a_k = \begin{cases} \tilde{a}_k, & \tilde{a}_k \neq \text{HOLD} \wedge (E_k \vee M_k) \wedge \Gamma_k, \\ \text{HOLD}, & \text{otherwise.} \end{cases} \quad (10)$$

Every threshold in this decision is computed before appending the current score. The procedure therefore has no end-of-horizon catch-up rule and never spends a budget token merely to reach a quota.

## 4.2 Causal change signal

Goals released in the current trace window are binned into a  $4 \times 4$  spatial histogram with a 0.5 pseudocount per bin. Let  $p_k^f$  average the two most recent non-missing windows and let  $p_k^s$  average up to six preceding non-missing windows. The normalized change score is

$$z_k = \text{JS}(p_k^f, p_k^s) / \log 2 \in [0, 1], \quad (11)$$

and is zero when either side is unavailable. Thresholds are computed from past scores before appending  $z_k$ , so future tasks, latent phases, and future rewards cannot influence an earlier action.

## 4.3 Context catalog and recall

For each generated graph  $j$ , CARR stores the causal  $4 \times 4$  distribution  $c_j$  of active goals at generation time. With current context  $c_k$ ,

$$d_{k,j} = \text{JS}(c_k, c_j) / \log 2. \quad (12)$$

Let  $a$  be the active generation and  $j^*$  the nearest inactive catalog item. Recall is available only if

$$d_{k,j^*} \leq 0.05 \quad \text{and} \quad d_{k,a} - d_{k,j^*} \geq 0.02. \quad (13)$$

The controller first proposes an available historical match; otherwise it generates when maintenance is due, holds if active context still matches, or proposes a fresh generation. Maintenance is due after 25 guidance-age windows when  $z_k \leq 0.20$ .

## 4.4 State transitions and accounting

The three realized operations have different state semantics. HOLD reuses a defensive copy of the active raw tensor; neither identifier nor resource counter changes. REACTIVATE loads an immutable tensor from the catalog. Its generation ID is preserved, but a new installation version is issued so subsequent route builds can be attributed to the reinstallation. GENERATE invokes  $F_\theta$ , allocates both a new generation ID and a new installation version, and stores the raw output with context  $c_k$ . In all three cases the environment applies the same invalid-edge mask and normalization after selection, so recall replays the same raw artifact rather than a separately transformed approximation.

The switch counter is incremented only by an effective reactivation or fresh generation; the generator counter is incremented only by a fresh generation. Both post-bootstrap counts have ceiling  $B_{25}$ . This ceiling is not a six-call hard budget: CARR’s call count is an outcome and reaches 10 in the confirmation matrix. Comparisons with G4/G5 schedules therefore bracket its observed mean but are not per-run call-matched trials.

The controller operates on 16-bin distributions. With catalog size  $|M_k|$ , nearest-context lookup takes  $O(16|M_k|)$  distance work per decision; the frozen cap limits the catalog to at most the bootstrap graph plus 25 fresh generations. The catalog stores raw guidance tensors, so its dominant storage is  $O(|M_k|H_m W_m)$ . No gradient step, checkpoint update, or planner modification occurs online. We report this structural complexity only; because the experiment did not collect valid serial timing, it is not converted into a wall-clock speedup claim.

## 5 EXPERIMENTAL DESIGN

### 5.1 Frozen backbone and run configuration

The guidance backbone is a pinned OnlineGGO-style convolutional checkpoint with 3,084 learned values. Its six-channel spatial input passes through a  $6 \rightarrow 32 \times 3 \times 3$  convolution, a  $32 \rightarrow 32 \times 1 \times 1$  convolution, and a  $32 \rightarrow 4 \times 1 \times 1$  output layer, with the checkpoint’s batch normalization behavior preserved exactly. The resulting four channels assign raw costs to the four directed grid moves. Inference runs without gradient updates. Before the experiment, an independent loader was required to match the pinned implementation bit-for-bit on a deterministic nontrivial input, and the checkpoint file and effective float32 parameter vector were separately hashed.

The downstream simulator uses the same compiled GPIBT configuration in every arm. Both warehouse layouts are  $33 \times 57$  grids. We evaluate approximately 20% and 35% traversable-cell occupancy, producing the agent counts in Table 1. Every run first executes 200 warm-up steps, then scores 2,000 steps in 100 windows of 20. The mandatory bootstrap generation is installed before the first scored window. All later guidance decisions occur at window boundaries, whereas route construction and task completion continue inside the simulator at the original step resolution.

This freeze defines the scope of causal attribution. Checkpoint weights, normalization, map, planner parameters, starting positions, and complete task tape are identical across the eight paired policies for a given root and scenario. The only permitted intervention is the publication operation at a decision boundary. Safety outcomes and route-version traces are independently replayed from

**Table 1: Frozen scenarios.**

| Scenario     | Map family | Agents |
|--------------|------------|--------|
| narrow_r020  | narrow     | 218    |
| narrow_r035  | narrow     | 382    |
| regular_r020 | regular    | 255    |
| regular_r035 | regular    | 447    |

each emitted trajectory rather than trusted solely from the controller’s internal counters.

## 5.2 Scenarios and workloads

All arms use the same frozen 3,084-parameter OnlineGGO-style CNN checkpoint and pinned GPIBT simulator. Table 1 crosses two warehouse maps with two densities. Each scenario is run with stationary, abrupt, and recurrent task streams.

Forty root clusters were fixed before execution. The complete matrix contains 8 policies  $\times$  40 roots  $\times$  4 scenarios  $\times$  3 workloads = 3,840 runs. Every policy has 480 paired cells, but the root—not a cell, window, task, or agent—is the independent unit.

Before Experiment B, a 960-run, ten-root A2 experiment on the same backbone failed its pre-specified 1% non-inferiority test: its mean effect was  $-0.4009\%$ , with a 95% interval of  $[-1.0834\%, 0.2584\%]$  and shifted one-sided  $p = .06836$ . Simulations that resampled centered A2 root residuals informed B’s 40-root sample-size choice, but those simulations were design calculations rather than B evidence. B uses forty fresh roots disjoint from A2; the studies are reported separately and are never pooled.

## 5.3 Publication policies

Table 2 lists the frozen controller family. CARR-NoRECALL maps to context\_no\_reactivation\_B25 and changes only recall availability. EXACT-B25 provides the high-call reference.

The comparator set isolates distinct explanations for the focal operating point. BOOTSTRAP tests whether any online publication is needed after one learned graph has been installed. EXACT-G4 and EXACT-G5 represent simple, fully reproducible sparse periodic schedules near CARR’s observed call range; RANDOM-G5 retains the six-call total but randomizes the five later decision indices with a root-keyed schedule. JS-G5 asks whether the release-change score alone is sufficient when capped at five post-bootstrap calls. CARR-NoRECALL preserves the proposal ordering, thresholds, guards, and B25 ceilings but treats historical recall as unavailable, making it the mechanism ablation. Finally, EXACT-B25 spreads 25 post-bootstrap calls evenly across the 99 eligible decisions and supplies the high-call throughput reference.

These are publication-policy comparisons within one fixed backbone, not a leaderboard across different LMAPF solvers. In particular, changing from GPIBT to RHCR or another planner would alter the data plane at the same time as the control policy and would no longer identify the effect of refresh timing. The external-validity question of whether the same controller transfers across planners is therefore left separate from the internal comparison reported here.

## 5.4 Frozen inference and integrity

For focal policy  $A$  and comparator  $B$ , the pre-specified root effect averages the 12 scenario–workload cells equally:

$$d_i(A, B) = \frac{1}{12} \sum_{c=1}^{12} \frac{C_{A,i,c} - C_{B,i,c}}{C_{B,i,c}}. \quad (14)$$

The equal average gives each map–density–workload cell the same weight; high-throughput cells do not dominate merely because they complete more tasks. A root jointly determines exogenous starting positions, task randomness, and the paired policy schedule across all twelve cells. Outcomes within that root—including cells, windows, agents, and completed tasks—are therefore correlated. Treating the 3,840 runs or the millions of agent decisions as independent observations would severely overstate precision. We first reduce each root to  $d_i$  and perform every inferential resampling or sign assignment at the whole-root level.

We report the mean of forty root effects, root win/tie/loss counts, and a 10,000-sample whole-root percentile-bootstrap 95% interval. Superiority uses one-sided exact paired sign-flip tests—computed exactly over the  $2^{40} \approx 1.1 \times 10^{12}$  root sign assignments by a meet-in-the-middle enumeration—with Holm step-down correction over six frozen contrasts. The separate test against EXACT-B25 uses a  $-1\%$  non-inferiority margin and requires all three frozen conditions: mean effect at least  $-0.5\%$ , interval lower bound strictly above  $-1\%$ , and shifted one-sided exact  $p < .05$ .

Each arm ran in a fresh process. Append-only ledgers bind the arm, process instance, run UUID, paired-input fingerprints, safety state, and output hash. The audit requires the exact Cartesian product, one start and completion per arm, zero hidden retries or deletions, matched exogenous inputs, valid budget schedules, and zero collisions, swaps, invalid moves, or planner timeouts. The controller definitions, matrix, estimands, inference, stopping rule, and audit gates were internally pre-specified and frozen before execution, and the forty roots were fixed before any fresh-root run; we do not describe this internal freeze as public preregistration.

## 6 RESULTS

### 6.1 Integrity and primary non-inferiority result

All 3,840 runs passed the integrity audit. The pre-specified primary B objective was not met. CARR completes 49.29 fewer tasks than EXACT-B25 on the grand mean. Its root-level relative effect is  $-0.8006\%$ , with a 95% interval of  $[-1.1935\%, -0.3966\%]$  and a shifted exact one-sided  $p = .1693$ . All three pre-specified conditions fail: the mean effect is below  $-0.5\%$ , the interval lower bound is not strictly above  $-1\%$ , and the shifted test does not pass  $.05$ . The frozen 1% non-inferiority criterion therefore fails. The interval lies entirely below zero, indicating a measurable throughput deficit on this benchmark; it does not establish that CARR preserves throughput within 1% of dense refresh.

The point estimate also varies by map: it is  $-0.41\%$  on the narrow map and  $-1.19\%$  on the regular map, reaching  $-1.71\%$  in the dense regular\_r035 scenario. By workload the deficit ranges from

**Table 2: Evaluated publication policies. Counts include the mandatory bootstrap call where stated.**

| Paper name    | Frozen implementation       | Publication contract                        |
|---------------|-----------------------------|---------------------------------------------|
| BOOTSTRAP     | bootstrap_only              | Bootstrap once, then hold                   |
| EXACT-G4      | exact_even_G4               | Bootstrap + 4 even calls (5 total)          |
| EXACT-G5      | exact_even_G5               | Bootstrap + 5 even calls (6 total)          |
| RANDOM-G5     | random_G5                   | Bootstrap + 5 root-keyed random calls       |
| JS-G5         | js_cap_G5                   | Causal JS trigger, at most 5 post calls     |
| CARR-NoRECALL | context_no_reactivation_B25 | CARR logic without historical recall        |
| CARR          | context_memory_B25          | Hold/reactivate/generate under B25 ceilings |
| EXACT-B25     | exact_even_B25              | Bootstrap + 25 even calls (26 total)        |

−0.31% (stationary) to −1.33% (abrupt). These pre-specified decompositions illustrate heterogeneity but do not replace the failed aggregate test.

## 6.2 Throughput and generator calls

The pre-specified secondary analyses characterize the operating point after the failed primary test. Table 3 reports mean throughput and generator calls. CARR completes 4,539.62 tasks with 5.46 calls on average. Its empirical call distribution is sparse but variable: median 5, 90th percentile 8, and maximum 10.

The sample-mean frontier is BOOTSTRAP, EXACT-G4, CARR, CARR-NoRECALL, and EXACT-B25. CARR point-dominates the three six-call G5 controls on sample means, but this does not imply pairwise statistical superiority. Relative to EXACT-B25, calls decrease from 26 to 5.46, a descriptive reduction of 79.01%. No corresponding run-time reduction is claimed.

## 6.3 Which secondary sparse comparisons are supported?

Table 4 reports the six frozen superiority contrasts. After Holm correction, CARR is significantly superior to all five frozen low-call comparators—BOOTSTRAP, EXACT-G4, EXACT-G5, RANDOM-G5, and JS-G5—including the strongest fixed six-call periodic schedule. It is not superior to its own CARR-NoRECALL ablation, whose interval crosses zero (Holm  $p = .925$ ); a confidence interval crossing zero is not evidence of equality.

## 6.4 What does historical recall contribute?

Against CARR-NoRECALL, CARR’s throughput effect is −0.108% (95% CI [−0.257%, 0.029%]; Holm  $p = .925$ ). Thus the experiment detects no throughput benefit from recall. The observed mean total-call count falls from 7.07 to 5.46 (22.8%), and post-bootstrap fresh generations fall from 6.07 to 4.46 (26.6%), while effective switches rise slightly from 6.07 to 6.20 because 1.74 historical graphs are reactivated on average. These call differences are descriptive; no call-difference significance test was pre-specified. The supported mechanism statement is therefore narrow: recall substitutes for some fresh generations, not that it improves throughput.

## 7 DISCUSSION AND LIMITATIONS

*When should guidance be refreshed?* The results favor neither always holding nor refreshing at every opportunity. Relative to

bootstrap-only reuse, context-aware publication yields +4.42% throughput; relative to a 26-call schedule, it spends 79.01% fewer calls but carries a measurable throughput deficit whose 95% interval lies entirely below zero yet extends past the −1% margin. Unlike the ten-root A2 study, the pre-specified secondary comparison with the fixed six-call even schedule now favors CARR after correction. The practical answer is therefore an operating curve rather than a universally best trigger: CARR is useful when generator calls are the constrained unit, but dense refresh remains the highest-throughput sample mean.

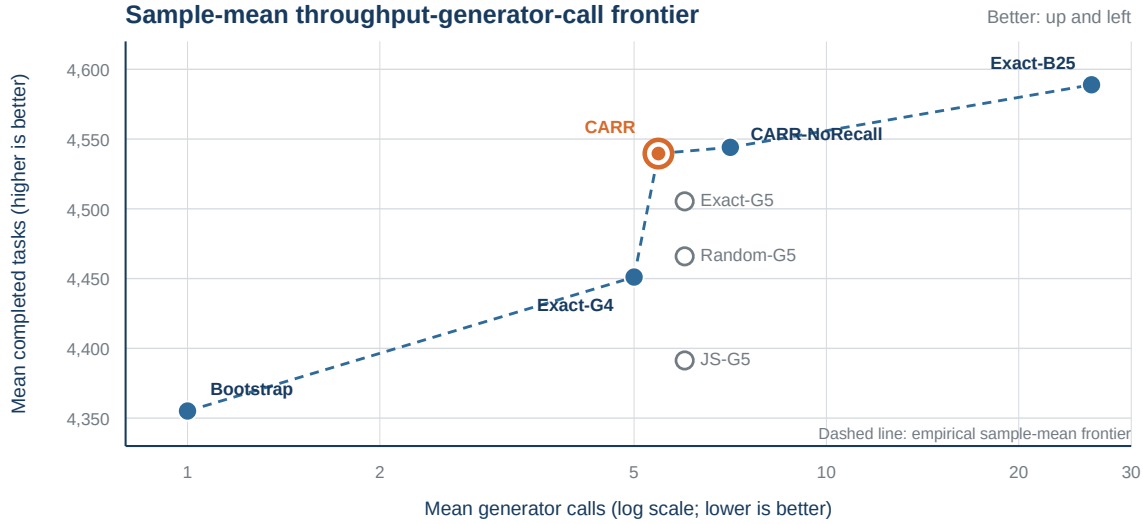
*Coordination-system relevance.* The unit controlled by CARR is a shared artifact that many agents adopt at different route-construction times. Explicit installation versions and route maturity make this delayed adoption auditable. This control-plane view is applicable to distributed agent systems in which an expensive shared policy, plan, or coordination signal is recomputed less frequently than local actions. Our evidence, however, is specific to the evaluated routing backbone.

*External validity.* The study fixes one small CNN checkpoint, one GPIBT-style backbone, two related warehouse maps, two densities, three synthetic workload families, and forty independent root clusters. It neither compares against external LMAPF systems nor establishes global SOTA. Generalization to other maps, generators, planners, or task models remains open.

*Resource interpretation.* Generator calls are logical counts. We did not obtain valid serial timing, GPU-time, energy, or financial-cost measurements. Concurrent experiment execution cannot be repurposed as timing evidence, so call reduction must not be read as proportional wall-clock acceleration.

*Statistical and Pareto uncertainty.* There are forty independent root clusters; 3,840 runs are not 3,840 independent replicates. Exact sign-flip inference is computed over  $2^{40} \approx 1.1 \times 10^{12}$  root sign assignments by meet-in-the-middle. The larger sample tightens the non-inferiority interval relative to A2, but the B interval still extends below the −1% margin while lying entirely below zero; increased precision does not establish non-inferiority. Frontier membership uses grand sample means and is not an uncertainty-aware Pareto-dominance test.

*Budget comparability and mechanism identification.* CARR is sparse on average but is not a hard G5 method; 24.0% of its runs exceed six calls. Moreover, the no-recall ablation has slightly higher mean



**Figure 2: Sample-mean throughput-generator-call operating points. The dashed line joins the empirical mean frontier; it is not an uncertainty-aware dominance claim. CARR is highlighted in orange.**

**Table 3: Sample-mean operating points. Frontier membership concerns the two point estimates (mean tasks, mean calls), not statistical dominance.**

| Policy        | Tasks   | Calls mean | Median | P90 | Max | Reactivations | Frontier |
|---------------|---------|------------|--------|-----|-----|---------------|----------|
| BOOTSTRAP     | 4355.08 | 1.00       | 1.0    | 1   | 1   | 0.00          | yes      |
| EXACT-G4      | 4451.10 | 5.00       | 5.0    | 5   | 5   | 0.00          | yes      |
| EXACT-G5      | 4505.35 | 6.00       | 6.0    | 6   | 6   | 0.00          | no       |
| RANDOM-G5     | 4465.94 | 6.00       | 6.0    | 6   | 6   | 0.00          | no       |
| JS-G5         | 4391.32 | 6.00       | 6.0    | 6   | 6   | 0.00          | no       |
| CARR-NoRECALL | 4544.06 | 7.07       | 7.0    | 10  | 13  | 0.00          | yes      |
| CARR          | 4539.62 | 5.46       | 5.0    | 8   | 10  | 1.74          | yes      |
| EXACT-B25     | 4588.90 | 26.00      | 26.0   | 26  | 26  | 0.00          | yes      |

**Table 4: Root-level effects of CARR minus each comparator. Positive values favor CARR. Intervals are whole-root bootstrap intervals.**

| Comparator    | Task $\Delta$ | Mean effect | 95% CI         | W/T/L   | Raw $p$               | Holm $p$              |
|---------------|---------------|-------------|----------------|---------|-----------------------|-----------------------|
| BOOTSTRAP     | +184.54       | +4.42%      | [3.75, 5.07]%  | 40/0/0  | $9.1 \times 10^{-13}$ | $5.5 \times 10^{-12}$ |
| EXACT-G4      | +88.52        | +2.17%      | [1.77, 2.59]%  | 37/0/3  | $7.3 \times 10^{-12}$ | $2.9 \times 10^{-11}$ |
| EXACT-G5      | +34.26        | +0.75%      | [0.32, 1.18]%  | 28/0/12 | $7.7 \times 10^{-4}$  | $1.5 \times 10^{-3}$  |
| RANDOM-G5     | +73.68        | +1.69%      | [1.27, 2.11]%  | 36/0/4  | $1.1 \times 10^{-9}$  | $3.4 \times 10^{-9}$  |
| JS-G5         | +148.30       | +3.36%      | [2.85, 3.87]%  | 39/0/1  | $1.8 \times 10^{-12}$ | $9.1 \times 10^{-12}$ |
| CARR-NoRECALL | -4.44         | -0.11%      | [-0.26, 0.03]% | 18/0/22 | .925                  | .925                  |

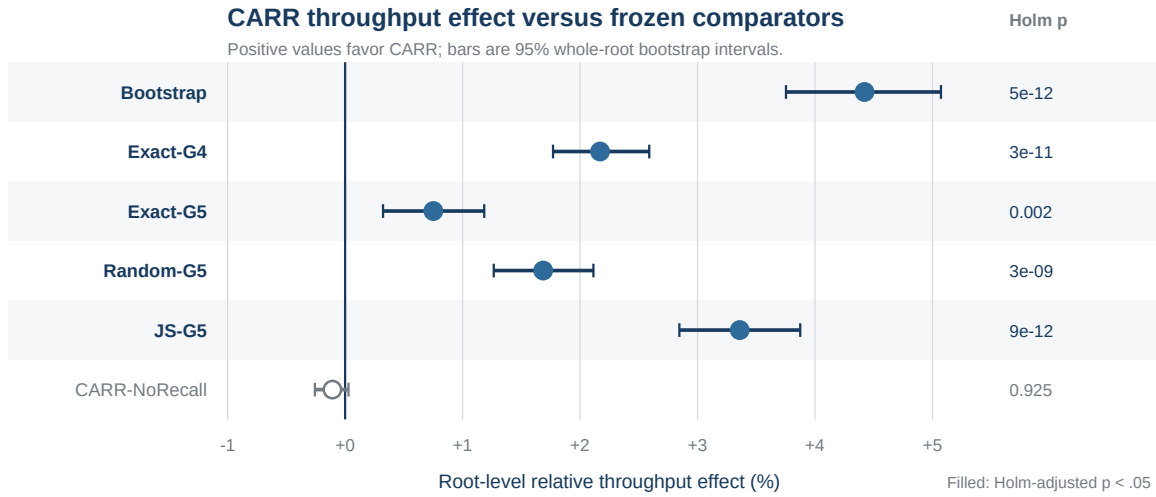
throughput. The evidence cannot attribute the operating point to memory alone.

*Analysis provenance.* The controller matrix, estimands, tests, and stopping rules were frozen before execution, and the forty roots were fixed before any fresh-root run. The emphasis on a Pareto operating point is a transparent post-validation narrowing after a stronger near-dense claim was not supported. We therefore report

the failed non-inferiority result in the abstract, results, and conclusion rather than presenting the descriptive frontier as the original confirmatory claim.

## ETHICAL AND SOCIETAL CONSIDERATIONS

This study evaluates guidance-refresh policies entirely in simulation using synthetic task streams and public benchmark maps; it



**Figure 3: Frozen root-level superiority contrasts for CARR. Filled points remain significant after Holm correction; a confidence interval crossing zero is not evidence of equality.**

involves no human participants, personal data, or deployment decisions. In a physical warehouse, incorrect guidance could reduce throughput or create operational risk even when the downstream planner enforces collision constraints. Deployment would therefore require site-specific safety validation, monitoring, and human override procedures; the proposed controller is not a substitute for a safety-certified motion-planning stack.

## 8 CONCLUSION

We studied when a fixed OnlineGGO-style guidance generator should be invoked within a GPIBT lifelong-routing backbone. Across 3,840 audited paired runs over forty independent roots, the primary 1% non-inferiority objective is not met: CARR has a  $-0.80\%$  mean throughput effect relative to 26-call dense refresh (95% CI  $[-1.19\%, -0.40\%]$ ). In the pre-specified secondary analyses, CARR uses 5.46 calls on average—79.01% fewer than dense refresh—significantly outperforms all five frozen low-call comparators, and lies on the sample-mean throughput-call frontier. Historical recall replaces some fresh generations without a detected throughput benefit. The resulting contribution is a bounded resource-throughput evaluation and an auditable low-call operating point, not a lossless replacement or global LMAPF SOTA.

## REFERENCES

- [1] Zhe Chen, Daniel Harabor, Jiaoyang Li, and Peter J. Stuckey. 2024. Traffic Flow Optimisation for Lifelong Multi-Agent Path Finding. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 18 (2024), 20674–20682. <https://doi.org/10.1609/aaai.v38i18.30054>
- [2] Liron Cohen, Tansel Uras, and Sven Koenig. 2015. Feasibility Study: Using Highways for Bounded-Suboptimal Multi-Agent Path Finding. In *Proceedings of the Eighth Annual Symposium on Combinatorial Search*. 2–8. <https://doi.org/10.1609/socs.v6i1.18363>
- [3] Jihee Han and Yoonho Seo. 2018. Path Regeneration Decisions in a Dynamic Environment. *Information Sciences* 450 (2018), 39–52. <https://doi.org/10.1016/j.ins.2018.03.035>
- [4] Kohei Honda, Ryo Yonetani, Mai Nishimura, and Tadashi Kozuno. 2024. When to Replan? An Adaptive Replanning Strategy for Autonomous Navigation using

- Deep Reinforcement Learning. In *2024 IEEE International Conference on Robotics and Automation*. 6650–6656. <https://doi.org/10.1109/ICRA57147.2024.10611474>
- [5] M. Renee Jansen and Nathan R. Sturtevant. 2008. Direction Maps for Cooperative Pathfinding. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment* 4, 1 (2008), 185–190. <https://doi.org/10.1609/aiide.v4i1.18695>
- [6] Jiaoyang Li, Andrew Tinka, Scott Kiesel, Joseph W. Durham, T. K. Satish Kumar, and Sven Koenig. 2021. Lifelong Multi-Agent Path Finding in Large-Scale Warehouses. *Proceedings of the AAAI Conference on Artificial Intelligence* 35, 13 (2021), 11272–11281. <https://doi.org/10.1609/aaai.v35i13.17344>
- [7] Hang Ma, Jiaoyang Li, T. K. Satish Kumar, and Sven Koenig. 2017. Lifelong Multi-Agent Path Finding for Online Pickup and Delivery Tasks. In *Proceedings of the 16th International Conference on Autonomous Agents and MultiAgent Systems*. 837–845.
- [8] Keisuke Okumura, Manao Machida, Xavier Défago, and Yasumasa Tamura. 2019. Priority Inheritance with Backtracking for Iterative Multi-Agent Path Finding. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*. 535–542. <https://doi.org/10.24963/ijcai.2019/76>
- [9] Paulo Tabuada. 2007. Event-Triggered Real-Time Scheduling of Stabilizing Control Tasks. *IEEE Trans. Automat. Control* 52, 9 (2007), 1680–1685. <https://doi.org/10.1109/TAC.2007.904277>
- [10] Hongzhi Zang, Yulun Zhang, He Jiang, Zhe Chen, Daniel Harabor, Peter J. Stuckey, and Jiaoyang Li. 2025. Online Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 14 (2025), 14726–14735. <https://doi.org/10.1609/aaai.v39i14.33614>
- [11] Yulun Zhang, Varun Bhatt, Matthew C. Fontaine, Stefanos Nikolaidis, and Jiaoyang Li. 2026. Optimization of Edge Directions and Weights for Mixed Guidance Graphs in Lifelong Multi-Agent Path Finding. *arXiv:2602.23468 [cs.MA]*
- [12] Yulun Zhang, He Jiang, Varun Bhatt, Stefanos Nikolaidis, and Jiaoyang Li. 2024. Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. 311–320. <https://doi.org/10.24963/ijcai.2024/35>

## A EXECUTION AUDIT SUMMARY

Experiment B used forty fresh roots disjoint from the prior ten-root A2 experiment. A2 informed sample-size planning but contributes no root, estimate, interval, or p-value to B. Earlier in the program, one launcher execution was discarded effect-blind after it treated legal operating-system PID reuse as duplicate process identity; no outcome or method ranking from it entered estimation or interpretation. In B, 11 legal bare-PID reuses yielded 3,829 distinct

**Table 5: Frozen Experiment B integrity audit.**

| Audit item                    | Result      |
|-------------------------------|-------------|
| Required runs                 | 3,840/3,840 |
| Started/completed attempts    | 3,840/3,840 |
| Unique process instances      | 3,840       |
| Unique canonical run UUIDs    | 3,840       |
| Unique bare PIDs (non-gating) | 3,829       |
| Method failures/retries       | 0           |
| Planner timeouts              | 0           |
| Selective/timeout deletions   | 0           |
| Safety, pairing, budgets      | all pass    |

PIDs, while all 3,840 host–PID–start-time process instances and run UUIDs remained unique; bare PID alone was therefore not an integrity gate.

**B REPRODUCIBILITY PROVENANCE**

Before execution, the Experiment B protocol, configuration, implementation freeze, environment attestation, and preflight evidence were sealed by SHA-256. After execution, the integrity report, machine-readable analysis, human-readable report, and complete archive were separately content-hashed. The final analysis artifact is `reports/same_call_confirmation_b_analysis.json` with digest `cceab3d62390d6a1acd1c5ee3e78ff2048851a20c448796f968f212db0fb428b`; the complete run archive has digest `6acf741f7ac460458558f1892030b46072fa438e59f639196fee394a5d3658e0`. The anonymized artifact will include the complete manifests, source hashes, ledger checks, analysis and figure-generation scripts, and controller-name mapping. Its machine-readable analysis also reports all forty root effects, their median, map/workload/scenario decompositions, and leave-one-root-out means committed by the frozen protocol.